

Project), and a **Build Target**—you can choose from 2.2, 3.2, 4.0, etc. I chose 2.2. Specify an **Application Name**—I used *MyFirstApplication*; then a **Package Name**—the name must start with *com* (e.g., *com.firstProject*), and an **Activity Name**—the Java file name (I used *MyFirstActivity*). Click **Finish**. Now the new project should be shown in the Project Explorer, as in Figure 1.

To integrate the downloaded PhoneGap with this project, create a new directory named *libs*. Open your PhoneGap archive and from the *Android* folder, copy the *.jar* file. Paste it in the newly-created *libs* folder of your project. Under the project's *assets* folder, create a new *www* folder. Copy the *js* file from PhoneGap's *Android* folder into the new *www* folder. We also need to set the build path. Right-click on the project > **Build Path** > **Configure Build Path** > **Libraries** > **Add Jar** and add the PhoneGap *.jar* file. Now paste all the downloaded JQuery Mobile files in the *www* folder.

Let's move on to the implementation. We can embed our code in a simple HTML page in the *www* folder; just copy and paste the *index.html* file that we created earlier into the *www* project folder. Next, edit *MyFirstActivity.java* and add the following command:

```
import com.phonegap.DroidGap;
public class MyFirstActivity extends DroidGap
```

Replace *setContentView()* with *super.loadUrl("file:///android_asset/www/index.html")*; and save. To run it in the emulator, right-click on **Project** and **Run As** > **Android Application**. One more thing is needed to add the emulator: **Windows** > **AVD Manager** > specify any name > **Ok**. The running project in the emulator looks like what's shown in Figure 2. Congratulations! You are now a mobile developer.

JQuery Mobile special features

I'm sure that you're anxious to see the special features in action; so let's try each one out in the following sections.

List-views: Let's just add some code to our template inside the *data-role="content"* div:

```
<ul data-role="listview" data-inset="true" data-filter="true">
  <li><a href="#">Agriculture</a></li>
  <li><a href="#">Animal</a></li>
  <li><a href="#">Astronomy</a></li>
```



Figure 1: Project structure

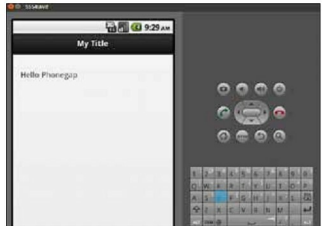


Figure 2: Hello World

```
<li><a href="#">Building</a></li>
<li><a href="#">Business</a></li>
</ul>
```

In this code, the three attributes used for creating the list view are:

1. *data-role="listview"*, which will create the view as a list, with the ** items;
2. *data-inset="true"* which will provide a bit of margin and rounded corners within the content area; and...
3. *data-filter="true"* which will create a search bar.

Running the project now will give results like in Figure 3.

This was a basic list-view; other kinds will customise your list accordingly. You can create various list-views using the following:

- Numbered list: Uses the ** ordered list tag.
- List dividers: Which can be implemented using *data-role="list-divider"* or using *data-autodividers="true"*.
- List with count bubbles: Wraps the item with a *class ui-li-count*:

```
<li><a href="#">Agriculture</a></li>
<span class="ui-li-count">86</span>
```

- List with thumbnails:

```
<li><a href="#">Agriculture
  
</a></li>
```

- List with icons: For an icon, the item should be bound with *class ui-li-icon*:

```
<li><a href="#">Agriculture
  
</a></li>
```